
A Double-categorical Structure for Interaction and Learning with Agentic AI

Theodore Papamarkou^{*1} Yiyang Jia^{*2} Anton Ayzenberg³ Manuel Baltieri⁴
Luciano Floridi⁵ Marius Furter⁶ Xuexing Lu⁷ Sridhar Mahadevan⁸ Kelin Xia⁹
Junchi Yan¹⁰ Zhaoyang Yu¹¹ Yivan Zhang¹² Mustafa Hajij¹³

¹PolyShape Inc. ²Tokyo City University ³Noeon Research ⁴Araya Inc. ⁵Yale University
⁶University of Zurich ⁷Zaozhuang University ⁸UMass Amherst ⁹Nanyang Technological University
¹⁰Shanghai Jiao Tong University ¹¹DeepWisdom Inc. ¹²The University of Tokyo
¹³University of San Francisco

Abstract

To be completed.

1 Introduction

To be completed. Cite Gavranović et al. [2024].

1.1 Conceptual Motivation for Symmetric Monoidal Double Categories

1.2 Double SMCs as a Refinement of Tile Systems in AI Systems

Tile systems, as introduced by Gadducci and Montanari Bruni et al. [2002], provide a powerful and highly general framework for the modular operational semantics of concurrent systems. A tile specifies how a *configuration may be rewritten in a given context, producing an observable effect*, and the calculus supports *both sequential and parallel composition of such rewritings*. This two-dimensional structure—configurations evolving under observable interactions—anticipates many of the diagrammatic features formalized in double categories.

Tile systems provide a highly general, modular paradigm for the operational semantics of concurrent systems. Their key methodological contribution is to make explicit a two-dimensional semantic organization: configurations compose horizontally (e.g., via sequential and parallel composition), while computational evolution proceeds vertically by rule-triggered rewriting, producing observable effects. This separation is not merely a notational convenience; it reflects a structural necessity for open systems in which evolution is context-dependent and interaction is part of the semantics. In particular, tile formalisms enforce that a step of computation is meaningful only relative to a surrounding context and an interface-level observation/effect, which can then be composed with other steps by a disciplined pasting calculus.

From a categorical viewpoint, however, the tile formalism is most fruitfully understood as being organized around a *strict double-categorical backbone*. Concretely, tile systems distinguish two orthogonal dimensions of structure: a horizontal dimension capturing how configurations (or contexts) are composed and embedded along interfaces, and a vertical dimension capturing how configurations evolve by rewriting. A *tile* is precisely a two-dimensional cell with a rectangular boundary, relating a horizontal boundary (contextual embedding) and a vertical boundary (evolution), and tile computations are obtained by pasting such squares both vertically and horizontally. The existence of both pasting operations together with the interchange law is exactly the hallmark of a (strict) double

^{*}These authors contributed equally to this work.

Table 1: Structural comparison between tile systems and agentic/multi-agent frameworks.

Dimension	Tile-system setting	Agentic / multi-agent setting (this paper)
Basic unit	Process / configuration	Agent + environment (and their interfaces)
Behavioural primitive	Rule-triggered rewriting (operational step)	Decision, policy, interaction, tool use
Context notion	Syntactic / interface context (structural embedding)	Environment, other agents, institutions / governance regimes
Observation / evaluation	Discrete events (e.g. signal/create/consume)	Quantitative or ordered evaluation (reward, loss, risk, confidence)
Primary goals	Correctness, reachability, concurrency semantics	Performance, learning, coordination, alignment/governance
Two-dimensional reading	State/configuration vs. observation/effect	Behaviour vs. evaluation (with compositional aggregation)

category: a category of horizontal 1-cells, a category of vertical 1-cells on the same objects, and a family of 2-cells (squares) equipped with compatible horizontal and vertical composition.

Importantly, a tile system is not *merely* an arbitrary strict double category. Rather, it is a *structured instance* of a strict double category, typically specified by a chosen signature of generators (basic configurations and basic observation/effect constructors), together with a set of generating 2-cells (the basic tiles/rules) and additional operational constraints that control which pasting steps are admissible and how observations are produced and propagated. In particular, tile logic is designed so that each rewriting step is meaningful only relative to a surrounding context and produces an explicit, event-like observation (e.g. signal creation/consumption, synchronization effects). This event-based observation structure is integral to the concurrency-theoretic applications of tile systems, where the primary semantic goals include correctness, reachability, and behavioural equivalence of concurrent processes under composition.

Although tile systems were originally introduced to model the operational semantics of concurrent processes, their underlying double-categorical structure is highly instructive for the agentic setting considered in this paper. At a conceptual level, **tile logic makes explicit a fundamental distinction between two orthogonal dimensions of system semantics: the horizontal dimension, which captures how components are composed and embedded into larger contexts, and the vertical dimension, which captures how systems evolve or compute within such contexts. This separation is not a technical artifact of concurrency theory, but a structural necessity for modeling open systems.**

Modern AI systems, and in particular agentic and multi-agent systems, are quintessentially open. **Models, agents, environments, tools and institutional constraints are composed along interfaces, while behaviour, decision-making and learning unfold relative to these compositions. As in tile systems, evolution cannot be meaningfully described in isolation from context, and context cannot be reduced to a static background.** The double-categorical perspective provides a principled way to keep these aspects distinct while allowing them to interact through well-defined 2-cells.

From this viewpoint, tile systems offer an important methodological precedent: they demonstrate that **a two-dimensional semantic calculus is essential to avoid collapsing composition and evolution into a single notion of morphism.**

The difference between tile system and our framework is listed in Table 1.

Especially, we have the following reasons to use similar framework: (symmetric) double categories in our framework. Note that the symmetric structure in our agentic double categories is not imposed ad hoc. Rather, it arises naturally from the modeling of interfaces and parallel composition. Both behavioural and evaluative processes admit a notion of parallel combination that is invariant under reordering, reflecting the fact that agents, environments, and evaluative regimes are composed without an intrinsic left–right bias. The resulting symmetry is therefore induced by the semantics of the modeled systems rather than assumed as an additional constraint.

- **Horizontal and vertical dimensions are structurally necessary for open systems.** Tile systems make explicit that once a system is open, the way components are composed into contexts and the way they evolve within those contexts cannot be conflated into a single notion of morphism. The horizontal dimension captures structural composition and interface embedding, while the vertical dimension captures dynamic evolution. This separation reflects a structural necessity rather than a modeling convenience. Modern AI systems exhibit the same openness: agents, models, environments, tools, and protocols are assembled along interfaces, while inference, interaction, and learning unfold relative to these assemblies. A double-categorical framework preserves this essential distinction.
- **System evolution is inherently contextual rather than absolute.** In tile systems, a rewrite step only occurs relative to a triggering context and produces observable effects that are themselves part of the system interface. This highlights that evolution is never unconditional. The same principle applies to agentic systems: actions, decisions, and learning updates are always relative to an environment, an information state, and a regime of evaluation or constraint. The two-dimensional cell structure captures this dependence explicitly, preventing evolution from being modeled as a context-free transition.
- **Two-dimensional structure prevents semantic collapse between behaviour and evaluation.** Tile systems demonstrate that collapsing composition and evolution into a single dimension leads to a loss of semantic resolution. An analogous collapse occurs in AI modeling if behavioural processes and evaluative criteria are forced into a single categorical structure. Behavioural descriptions alone obscure why actions are acceptable or optimal, while evaluative descriptions alone lose contact with the processes they assess. Maintaining orthogonal dimensions for behaviour and evaluation preserves their distinct roles while allowing controlled interaction through 2-cells.
- **Agentic AI systems, like tile systems, are fundamentally open and modular.** Tile logic was designed to model systems whose components can be embedded, recomposed, and reused across contexts. This modularity mirrors the architecture of contemporary AI systems, where neural networks, agents, tool-augmented models, and institutional mechanisms function as open components. The horizontal composition mechanisms of tile systems provide a conceptual precedent for treating agentic systems as compositional assemblies rather than monolithic entities.
- **Local, composable semantic units are essential for scalable reasoning.** Each tile represents a local, composable fragment of causal structure that can be independently specified and later combined to yield global system behaviour. This locality principle is equally important in agentic settings. Learning rules, critics, overseers, and coordination mechanisms are typically defined locally but acquire meaning through composition. Modeling such components as 2-cells supports modular reasoning without imposing a global notion of optimality or equilibrium.
- **Double-categorical structure supports coherent modeling across scales.** Tile systems illustrate how a single two-dimensional formalism can describe both individual rewrite steps and the emergent behaviour of large concurrent systems. This scalability is crucial for agentic AI, where micro-scale mechanisms such as layers, policies, or update rules must be related to macro-scale phenomena including multi-agent coordination, governance, and institutional constraints. A double-categorical framework provides a uniform language for expressing such cross-scale relationships.

Following are some counterexamples.

Counterexample (collapse of openness). Suppose behaviour and composition are modeled within a single monoidal category, with morphisms representing both structural embedding and dynamic evolution. In such a setting, there is no principled way to distinguish changes induced by context from changes induced by internal dynamics. As a result, the same morphism may ambiguously represent either a reconfiguration of interfaces or a behavioural transition. This collapse prevents a faithful representation of open systems, where embedding into a larger context should not be conflated with state evolution. Tile systems avoid this ambiguity by separating these dimensions, and the same separation is required in agentic modeling.

Counterexample (loss of evaluative compositionality). Consider modeling an agent solely by a category of behavioural processes, with evaluative quantities such as losses or rewards encoded as at-

tributes of morphisms. In this approach, composing behaviours implicitly composes evaluations in an ad hoc manner, making it unclear how multiple evaluative regimes—such as task performance, safety constraints, and normative oversight—should interact. This leads either to arbitrary aggregation rules or to an implicit global objective. By contrast, separating behaviour and evaluation into orthogonal dimensions ensures that evaluative structure remains compositional and explicitly governed by its own algebraic laws.

Counterexample (globalization of semantics). If agentic semantics are defined only at the level of entire systems—e.g. via global optimality conditions or equilibrium concepts—then local components such as learning rules, critics, or institutional constraints cannot be reasoned about independently. Any modification to a local mechanism requires re-deriving global properties of the entire system. Tile systems demonstrate that local, composable semantic units are necessary to avoid this brittleness. Agentic double categories retain this advantage by treating agentic components as local 2-cells whose effects propagate through composition.

For these reasons, while tile systems treat observations as discrete events, we treat evaluation as first-class horizontal structure, typically modeled by enrichment. Concretely, we take behavioural processes to live in a vertical category (e.g., parametrized maps for differentiable computation, Markov kernels for stochastic interaction, or other behavioural process categories), while evaluative processes are modeled horizontally as enriched profunctors, relations, or other quantitative transformers valued in a monoidal poset/quantale. Two-cells then witness coherence between behaviour and evaluation: they assert that a behavioural component is admissible, optimal, or otherwise compatible with an evaluative regime, and they compose both vertically (temporal/causal evolution) and horizontally (aggregation and propagation of evaluation). This is especially crucial in multi-agent settings, where agents are not merely concurrent processes but entities subject to heterogeneous and interacting evaluative regimes. Rewards, critics, norms, and institutional constraints may apply asymmetrically to different agents, and their interaction must be represented compositionally rather than collapsed into a single global objective.

In summary, tile systems provide an important methodological precedent: they demonstrate that two-dimensional semantics is essential for open, compositional system modeling. Our framework can be seen as a principled extension of this idea to agentic AI, reinterpreting the horizontal dimension from event-based observation to quantitatively enriched evaluation in order to capture learning, coordination, and governance phenomena that are central in modern agentic and multi-agent systems.

In short, the reason we use double category instead of tiles is listed as follows.

- **Event vs. Value:** Tile observations are event-like; agentic evaluation is typically numeric/ordered and requires principled aggregation—hence enrichment.
- **Concurrency vs. Agency:** Tile systems target operational correctness and concurrency equivalence; agentic systems target learning, coordination, and governance, where evaluation is first-class.
- **Multi-agent heterogeneity:** Multi-agent AI involves heterogeneous evaluative regimes (rewards, critics, norms) across agents; this is naturally expressed by horizontal evaluative composition rather than event traces.

2 Position

Modern machine learning practice is dominated by variations of unstructured combinatorial search. At the architectural level, researchers and engineers wire together layers, attention blocks, memory modules, tools, and feedback connections, guided by collective intuition and empirical trial-and-error. At the agentic level, the same pattern reappears; multi-agent systems, LLM-based agents, and orchestration frameworks are assembled as graphs of interacting components (such as planners, critics, tool-callers, simulators, environments) again tuned by intuition and trial-and-error, yet the “wires” in these graphs often rely on prompt engineering and string parsing that obscure the true nature of the interaction. Across both scales, the design space is discrete and structured, but we lack a shared formal calculus in which to describe that structure, compare designs, and systematically explore alternatives.

In practice, the community has already made gestures toward such a calculus through diagrams. Neural network papers routinely include box-and-arrow schematics. Work on multi-agent systems

and orchestration frameworks does the same, drawing graphs of agents, channels, and feedback loops. However, these diagrams are informal. They rarely type their interfaces, they hide crucial distinctions (for example, between tensor data flow, gradient backpropagation, and causal control flow), and they are not machine-readable. As a result, they do not support automated verification, compositional reasoning, reuse of architectural motifs, or principled comparison between different models and agentic systems. AutoML and meta-learning methods operate on top of this informality, treating architectures as opaque objects rather than as structured composable entities. Moreover, these diagrams rarely distinguish between the structure that determines what the system does (its behaviour) and the structure that determines how it is trained, rewarded, or constrained (its evaluation).

Our position is that category theory should serve not merely as a uniform language for “models”, but as a *structural theory of agentic systems* that treats behaviour and evaluation as first-class, compositional objects. We advocate a two-dimensional diagrammatic calculus that distinguishes behavioural processes from evaluative processes and takes agents to be components that couple the two. Developing and adopting such a calculus, which generalises categorical deep learning from static architectures to closed-loop interaction and evaluation, is a necessary step toward principled design, comparison, and governance of contemporary AI systems.

In this calculus, one dimension captures behavioral processes (neural architectures, policies, environments) and the other captures evaluative processes (losses, rewards, critics, norms, meta-learning), and an agent is a component that couples the two. In this setting, a neural network with gradient descent, a reinforcement-learning (RL) agent in an environment, and an orchestrated multi-agent system all appear as instances of the same underlying agentic diagram, composed by the same operations. We refer to diagrams in this calculus as *agentic diagrams* and formalize them in Section 3 as a two-dimensional categorical structure.

This position unifies the “micro-scale” of network design with the “macro-scale” of agentic AI. At the micro-scale, agentic diagrams can describe internal computational pipelines, that is, how data flows through layers, how gradients and other learning signals propagate, how modules such as memory, attention, and recurrence interact within categories of differentiable manifolds or parametrized functions, while simultaneously recording the evaluative structure for training. At the macro-scale, the same syntax can describe agents, tools, evaluators, simulators, and environments as open processes in categories of Markov kernels or games with inputs and outputs, or in categories of typed communication protocols appropriate for LLM-based orchestration frameworks, wired together to form larger systems, together with the rewards, critics, and norms that evaluate them. Category theory supplies the glue; the same compositional principles and semantic structures apply in both regimes, so that a “layer”, an “agent”, and an “environment” can be treated as components in a shared agentic calculus rather than as conceptually disjoint entities.

Once models and agentic systems are expressed in a common categorical language, several benefits follow. First, model families become comparable. Two different architectures or orchestrations can be shown to be isomorphic (behaviorally equivalent), related by refinement, or differing only in specific subcomponents. Second, design patterns become reusable. A local motif, such as a feedback-controlled predictor or a critic-guided exploration module, can be abstracted as a categorical design pattern and reinserted into new architectures or agent configurations. Third, semantics becomes central. By interpreting agentic diagrams via functors into categories of differentiable, probabilistic, or causal structures, it becomes possible to reason formally about training, inference, interaction, causality, and failure modes (such as hallucinations or undesirable equilibria) within the same unified framework.

For agentic AI, this perspective shifts the focus from the empirical engineering of orchestration frameworks to a theory of interaction. Multi-agent games, control loops, agents, and self-evaluating systems can all be seen as instances of open, interconnected agentic diagrams whose behavior depends on their wiring and on the semantics assigned to the wires. Category theory, through its existing work on open systems, games, and learners, offers the conceptual framework to describe such interactions and a calculus for composing and transforming them. This opens the door to systematic exploration of agentic design spaces, searching not only over parameter values but also over diagram shapes, connectivity patterns, and higher-level compositions. It may even lead to the unification of the two fields of agentic systems and social systems under a single level of abstraction represented by multi-agent systems (irrespective of whether human, artificial or hybrid), a much needed development

in an increasingly digital world, and one that could also facilitate the translation of social explanatory mechanisms to AI.

In the remainder of the paper, we develop this position and illustrate it in several domains. Section 3 provides a minimal formalization of the proposed two-dimensional agentic calculus and shows how categorical deep learning and multi-agent RL instantiate it, and how the same pattern applies to contemporary LLM-based orchestration frameworks. Subsequent sections outline how agentic diagrams can serve as a categorical language for deep learning architectures, for agentic and multi-agent AI, and for related areas such as causal and probabilistic reasoning that support intelligent behavior. Our goal is not to replace existing modeling practices, but to show that many successful ideas in contemporary AI, both at the level of neural network architectures and at the level of agents, can be unified, expressed, related, and extended within a single categorical framework. By doing so, we argue that category theory does not merely “model intelligence” in the abstract, but offers a structural theory of agentic systems that can be instantiated from micro-scale deep learning architectures to macro-scale agentic AI systems.

3 A Two-dimensional Categorical Framework

In this section, we sketch a technical core for the position in Section 2. The informal idea is that modern AI systems carry two intertwined kinds of structure, namely behavioral structure, which is concerned with how information, control, and state flow through components such as layers, agents, environments, and protocols, and evaluative structure, which is concerned with how these components are trained, rewarded, constrained, or determined through losses, gradients, rewards, norms, and critics. We model this by a two-dimensional categorical calculus in which the vertical direction carries behavior and the horizontal direction carries evaluation. Individual components are 2-cells that couple the behavioral and evaluative structure. We first give an abstract definition, then instantiate it at the micro-scale of categorical deep learning and the macro-scale of multi-agent RL and agent orchestration.

3.1 Agentic Double Categories

We recall the notion of a strict double category in a form adapted to our setting.

Definition 1 (Strict double category). *A strict double category $\mathbb{A}$ consists of a class of objects $A, B, \dots$, a category $\mathbb{A}_v$ of vertical 1-cells $f : A \rightarrow B$ with objects those of $\mathbb{A}$, and a category $\mathbb{A}_h$ of horizontal 1-cells $p : A \Rightarrow B$ with the same objects. For each diagram of the form*

$$\begin{array}{ccc} A & \xrightarrow{p} & B \\ f \downarrow & & \downarrow g \\ A' & \xrightarrow{q} & B' \end{array}$$

with $f : A \rightarrow A'$, $g : B \rightarrow B'$ in $\mathbb{A}_v$ and $p : A \Rightarrow B$, $q : A' \Rightarrow B'$ in $\mathbb{A}_h$, there is a set of 2-cells α with this boundary, which we depict as a labeled square

$$\alpha : \begin{array}{ccc} A & \xrightarrow{p} & B \\ f \downarrow & \Downarrow \alpha & \downarrow g \\ A' & \xrightarrow{q} & B' \end{array}$$

or simply $\alpha : (p, f) \Rightarrow (q, g)$. There are vertical and horizontal composition operations on 2-cells, $\circ_v$ and $\circ_h$, and identity 2-cells $1_{f,p}$, such that vertical composition $\beta \circ_v \alpha$ is defined by pasting squares with matching horizontal boundaries, horizontal composition $\beta \circ_h \alpha$ is defined by pasting squares with matching vertical boundaries, both compositions are associative and unital, and the interchange law holds whenever both sides are defined:

$$(\beta_2 \circ_v \beta_1) \circ_h (\alpha_2 \circ_v \alpha_1) = (\beta_2 \circ_h \alpha_2) \circ_v (\beta_1 \circ_h \alpha_1).$$

In string-diagrammatic terms, a strict double category provides a calculus with two kinds of one-dimensional wires (vertical and horizontal) and two-dimensional boxes (2-cells) that can be composed both “top to bottom” and “left to right”.

Definition 2 (Agentic double category). *An agentic double category $\mathbb{A}$ equipped with a symmetric monoidal structure $(\mathbb{A}_v, \otimes, I)$ on the vertical category, which models parallel composition of behavioral processes, and a symmetric monoidal structure $(\mathbb{A}_h, \otimes, I)$ on the horizontal category, which models parallel composition of evaluative processes. The tensor on objects agrees in the two categories: for all objects A, B of $\mathbb{A}$ the tensor $A \otimes B$ in $\mathbb{A}_v$ and $\mathbb{A}_h$ is the same. The 2-cells and their compositions are required to be compatible with the tensor, so that tensoring squares side by side is functorial and satisfies the interchange law with both vertical and horizontal composition.*

We interpret objects A as interfaces, for example state spaces, observation and action types, or communication channels. Vertical 1-cells $f : A \rightarrow B$ are behavioural processes that transform interfaces, for example forward computation, environment dynamics or message passing. Horizontal 1-cells $p : A \Rightarrow B$ are evaluative processes, for example losses, reward flows, critics, norms or meta-parameters. A 2-cell $\alpha : (p, f) \Rightarrow (q, g)$ is an agentic component that specifies how behavioral and evaluative structure cohere: the behavioral process f together with the evaluative process p is implemented by a component that behaves as g under evaluation q . We call the resulting string diagrams agentic diagrams. Vertical composition of 2-cells corresponds to sequential composition of behavior while threading evaluation, and horizontal composition corresponds to sequential composition of evaluation while threading behaviour. The monoidal tensor $\otimes$ corresponds to parallel composition of both.

This abstract structure is deliberately modest. It does not prescribe a particular notion of learning, rationality or equilibrium. It only requires that agentic systems live in a compositional calculus where both behavior and evaluation are first-class and mutually constrain each other via 2-cells. The benefit is that different-looking systems, ranging from neural networks to multi-agent RL to LLM orchestration frameworks, can be expressed as diagrams in the same agentic double category (or in closely related ones). We now illustrate this at different scales.

3.2 Categorical Deep Learning as Micro-scale Agentics

We first show how categorical deep learning fits into this diagrammatic calculus. For concreteness, we fix a standard setting for differentiable computation.

Definition 3 (Parametrized maps). *Let $(\mathcal{C}, \otimes, I)$ be a symmetric monoidal category with finite products, modeling data types and differentiable maps, for example finite dimensional real vector spaces and smooth maps with Cartesian product.*

Convention. *Throughout, we adopt the standard associator*

$$\alpha_{A,B,C} : (A \otimes B) \otimes C \longrightarrow A \otimes (B \otimes C)$$

of a symmetric monoidal category. When composing parametrized maps, we implicitly use the coherence isomorphisms (associators and, when necessary, symmetries) of $\otimes$ to re-associate and reorder tensor factors.

The category $\text{Para}(\mathcal{C})$ of parametrized maps has the same objects as $\mathcal{C}$. A morphism $X \rightarrow Y$ is a pair (P, f) with P an object of $\mathcal{C}$, called the parameter object, and $f : P \otimes X \rightarrow Y$ a morphism in $\mathcal{C}$. Composition is defined by

$$(Q, g) \circ (P, f) := (P \otimes Q, g \circ (\text{id}_Q \otimes f) \circ \alpha_{Q,P,X}),$$

where $\alpha_{Q,P,X}$ denotes the canonical associator $(Q \otimes P) \otimes X \rightarrow Q \otimes (P \otimes X)$, with coherence isomorphisms implicitly applied to match tensor orderings, and identities are given by $(I, \lambda_X^{-1}) : X \rightarrow X$, where λ is the left unitor. This makes $\text{Para}(\mathcal{C})$ a symmetric monoidal category whose monoidal structure is inherited from $\mathcal{C}$. A morphism $(P, f) : X \rightarrow Y$ can be read as a neural network layer or block with parameter space P and forward map f .

We now define an agentic double category in which vertical structure encodes parametrized forward computation, horizontal structure encodes backward or evaluative flow (losses, gradients, critics), and 2-cells encode learning components that relate the two.

Definition 4 (Deep-learning agentic double category). *Fix a symmetric monoidal category $(\mathcal{C}, \otimes, I)$ as above and let $\mathbb{D}\mathbb{L}_v := \text{Para}(\mathcal{C})$ be the vertical category. Let $\mathbb{D}\mathbb{L}_h := \text{Para}(\mathcal{C})^{op}$ be the horizontal category (opposite category of $\text{Para}(\mathcal{C})$), so a horizontal 1-cell $p : X \Rightarrow Y$ is a parametrized map*

$(L, p^\sharp) : Y \rightarrow X$ in $\text{Para}(\mathcal{C})$, read as a mechanism for value or gradient propagation that flows backwards from outputs to inputs. Objects are those of $\mathcal{C}$ and represent activation or state types.

An elementary learning component from X to Y in this setting is given by the data

$$\mathbb{L} = (P, L, f : P \otimes X \rightarrow Y, b : L \otimes Y \rightarrow X, u : L \otimes P \rightarrow P)$$

where P is a parameter object, L is a “local value” or “error” object, f is a forward map, b propagates local evaluative information in L at the output back to the input space X (a generalized backward pass), and u updates parameters in response to local evaluative information. Such data can be organized into a 2-cell of a double category $\mathbb{DL}$ with vertical category $\mathbb{DL}_v$ and horizontal category $\mathbb{DL}_h$:

$$\begin{array}{ccc} X & \xrightarrow{(L, \text{id}_X)} & X \\ \mathbb{L} : (P, f) \downarrow & \Downarrow \mathbb{L} & \downarrow (P', f') \\ Y & \xrightarrow{(L, \text{id}_Y)} & Y \end{array}$$

Here, the updated vertical 1-cell (P', f') is induced by the parameter update u . Concretely, we take $P' = P$ and define

$$f' := f \circ (u \otimes \text{id}_X) \circ \alpha_{L, P, X}^{-1}.$$

The horizontal arrows (L, id_X) and (L, id_Y) thread through the local value object L , while the 2-cell $\mathbb{L}$ encodes how a step of learning updates parameters via u , and propagates values via b , according to the forward behavior f . Vertical composition of such squares corresponds to stacking layers into deep learning architectures and composing their learning rules, while horizontal composition corresponds to composing evaluative structure, for example by combining a task loss with a regulariser or a critic.

Formally, one can construct $\mathbb{DL}$ as the double category freely generated by such elementary components, or, more concretely, by adapting existing double-categorical constructions for lenses and open learners to the present parametrized setting. The important point for this paper is structural: a deep-learning model together with its training scheme is an agentic diagram in $\mathbb{DL}$, built from 2-cells that couple forward computation and evaluative feedback.

In this way, categorical deep learning provides the micro-scale primitives of our agentic calculus. Vertical arrows are parametrized neural architectures, while horizontal arrows and 2-cells formulate losses, gradients, and update rules under the same compositional language.

3.3 Multi-agent RL and Agentic Orchestration

We now provide an instantiation at the level of multi-agent reinforcement learning and agentic orchestration. As before, the same two-dimensional agentic pattern appears, but with a clear separation between *stochastic behavioural dynamics* and *evaluative structure*.

3.3.1 Agentic Structure: Behaviour and Evaluation

We first describe the agentic structure itself, independently of any specific learning paradigm. This separation allows us to treat agency as a structural notion, rather than one tied to a particular optimization or learning algorithm.

Definition 5 (Stochastic behavioural category). *Let $\mathcal{B} := \text{Stoch}$ denote the behavioural category in this context, where Stoch is the symmetric monoidal category of measurable spaces and Markov kernels. Objects are measurable spaces $X, Y, \dots$, and a morphism $k : X \rightarrow Y$ is a Markov kernel $k : X \rightarrow \mathcal{P}(Y)$. The monoidal tensor $\otimes$ is given by a suitable product of spaces, whenever such products are well-defined.*

Morphisms in Stoch are interpreted as stochastic behavioural processes, such as environment transitions, policies, communication channels, or message-passing dynamics.

Behaviour alone does not determine agency. To model rewards, values, norms, or credit-assignment mechanisms, we introduce an *evaluative enrichment* over the behavioural category.

Definition 6 (Evaluative enrichment). *Fix a symmetric monoidal quantale $(\mathcal{V}, \otimes, I)$, that is, a monoidal poset admitting arbitrary joins. An evaluative process is modeled as a $\mathcal{V}$ -enriched profunctor*

$$R : \mathcal{B}^{op} \times \mathcal{B} \longrightarrow \mathcal{V},$$

assigning to each pair of interfaces (X, Y) a value $R(X, Y)$ representing reward, cost, risk, or credit-assignment information.

Intuitively, $R(X, Y)$ measures how outcomes at Y are evaluated relative to structures or decisions at X .

Remark 1 (Engineering interpretations of $\mathcal{V}$). Different choices of the value quantale $\mathcal{V}$ correspond to different engineering interpretations of evaluative structure. The additive quantale $([0, \infty], +, 0)$ naturally models costs, risks, resource consumption, or degrees of constraint violation, where horizontal composition corresponds to accumulation. In contrast, the multiplicative quantale $([0, 1], \cdot, 1)$ models confidence, success probability, or trust, where composition reflects the decay of reliability along chains of decisions or interactions.

We now combine stochastic behaviour and evaluative enrichment into an agentic double category.

Definition 7 (Basic Agentic double category). *Define a basic agentic double category $\mathbb{A}$ as follows. Objects are interfaces, such as joint state spaces S , action spaces A_i , and composite products of these. (Observation or message interfaces, when relevant, are treated as part of the state or as auxiliary components of the behavioural processes, rather than as primitive objects.) We do not assume that all such products exist universally or are Cartesian; the tensor is understood as an abstract interface composition whenever meaningful.*

The vertical category $\mathbb{A}_v := \mathcal{B} = \text{Stoch}$ has 1-cells $f : A \rightarrow B$ representing stochastic behavioural processes, including environment dynamics, joint policy–environment steps, or message passing between agents.

The horizontal structure is given by $\mathcal{V}$ -enriched profunctors $R : A \Rightarrow B$ on $\mathcal{B}$, contravariant in A and covariant in B , encoding evaluative structure such as rewards, value functions, credit-assignment schemes, or normative constraints. Horizontal composition is given by enriched profunctor composition.

A 2-cell

$$\alpha : (R, f) \Rightarrow (R', g)$$

is an agentic component witnessing the compatibility between behavioural processes f, g and evaluative structures R, R' . Formally, α is a $\mathcal{V}$ -enriched transformation from R to the composite profunctor $R' \langle f, g \rangle$, expressing that evaluation is preserved or monotonically transformed along the behavioural change. Here $R' \langle f, g \rangle$ denotes the usual whiskering of an enriched profunctor along behavioural morphisms f and g . (We work at the level of a strict double category, up to the usual coherence isomorphisms for enriched profunctors. This choice ensures that vertical and horizontal composition of 2-cells is well-defined and satisfies the usual double-categorical laws.)

Semantically, a 2-cell asserts that a behavioural transition from f to g is *coherent* with respect to evaluation: the evaluative structure carried by R is preserved, or transformed in a controlled and admissible way, by the corresponding change to R' .

The qualifier *basic* emphasizes that $\mathbb{A}$ captures the structural core of agency independently of any particular learning paradigm. In the context of *multi-agent RL and agentic orchestration*, the term *multi-agent* should not be understood as requiring that each individual agent is itself a reinforcement-learning agent. While multi-agent reinforcement learning typically assumes homogeneous learning agents, contemporary LLM-based orchestration frameworks often involve heterogeneous agents, many of which do not perform reinforcement learning at all. The basic agentic double category $\mathbb{A}$ is designed to accommodate both settings within a single compositional framework, allowing reinforcement-learning agents and non-learning, rule-based, or LLM-driven agents to coexist and interact coherently.

3.3.2 Reinforcement Learning as an Instantiation

We now show how standard reinforcement learning fits into this agentic structure as a concrete instantiation, rather than as a defining primitive.

Single-agent Instantiation. A single reinforcement-learning agent interacting with an environment can be described by data

$$A = (\Theta, \pi : \Theta \otimes S \rightarrow \mathcal{P}(A), u : \Theta \otimes A \otimes R \rightarrow \Theta),$$

where Θ is an internal parameter or memory space, π is a (possibly stochastic) policy, and u is an update map driven by observations, actions, and realized rewards.

Fixing an environment kernel $E : S \otimes A \rightarrow \mathcal{P}(S \otimes R)$, the agent and environment together induce a vertical behavioural process

$$F : \Theta \otimes S \longrightarrow \mathcal{P}(\Theta \otimes S \otimes R)$$

in $\mathcal{B}$.

Evaluative structure, such as a critic, return estimator, or risk-sensitive objective, is represented separately as an enriched profunctor

$$R : (\Theta \otimes S)^{op} \times (\Theta \otimes S) \rightarrow \mathcal{V}.$$

By fixing $\mathcal{B} = \text{Stoch}$ together with a reward-oriented choice of evaluative structure, the basic agentic double category $\mathbb{A}$ specializes to a reinforcement-learning setting, which we denote by $\mathbb{RL}$.

These data can be organized into a 2-cell of $\mathbb{RL}$ of the form

$$\begin{array}{ccccc} & \Theta \otimes S & \xrightarrow{p} & \Theta \otimes S & \\ A : & F \downarrow & & \downarrow A & \downarrow G \\ & \Theta \otimes S \otimes R & \xrightarrow{q} & \Theta \otimes S \otimes R & \end{array}$$

where p and q are horizontal 1-cells encoding evaluative structure (e.g. critics, value relations, risk or normative constraints), and the 2-cell A expresses that the learning rule u couples the behavioural dynamics F, G with evaluation in a coherent manner. Note that profunctors are depicted as horizontal arrows in the sense of double categories. Here G denotes the post-update behavioural process induced after applying the learning rule u .

3.3.3 Multi-agent Composition and Orchestration

In the multi-agent case, we take tensor products of interfaces and parameters

$$(\Theta_1, \dots, \Theta_n, \pi_1, \dots, \pi_n, u_1, \dots, u_n),$$

and compose the corresponding 2-cells both vertically and horizontally. Vertical composition models temporal and causal interaction between agents and the environment, while horizontal composition models how evaluative structures, such as rewards, critics, norms, or institutional constraints, are aggregated and propagated across agents.

Diagrammatically, the multi-agent case is expressed by the same agentic 2-cell shape, with joint interfaces and parameters:

$$\begin{array}{ccccc} & \left(\bigotimes_{i=1}^n \Theta_i \right) \otimes S & \xrightarrow{p^{(n)}} & \left(\bigotimes_{i=1}^n \Theta_i \right) \otimes S & \\ A^{(n)} : & F^{(n)} \downarrow & & \downarrow A^{(n)} & \downarrow G^{(n)} \\ & \left(\bigotimes_{i=1}^n \Theta_i \right) \otimes S \otimes R^{(n)} & \xrightarrow{q^{(n)}} & \left(\bigotimes_{i=1}^n \Theta_i \right) \otimes S \otimes R^{(n)} & \end{array}$$

This diagram makes explicit that multi-agent interaction and orchestration do not introduce new agentic primitives, but arise from the same double-categorical structure by composition and scaling. Although phrased here in terms of Markov kernels and reinforcement learning, the construction is not tied to that setting. Other behavioural categories can replace Stoch to yield agentic double categories for LLM-based orchestration frameworks and other contemporary multi-agent systems.

Remark 2. Formally, coherence can be made precise by interpreting evaluative structures as order-enriched relations or profunctors and 2-cells as monotone transformations. We refrain from fixing a single formalization here, as the intended notion of evaluation varies across applications.

3.4 Unifying Scales in the Same Calculus

Definitions 4 and 7 illustrate the central technical point of our position. Although the underlying categories differ, differentiable maps for deep learning and stochastic kernels or other behavioural, such as communication processes for RL and agentic AI orchestration, both settings support the same two-dimensional agentic structure. Objects are interface types such as activations, states, observations, actions and messages. Vertical arrows are behavioral processes, such as layers and blocks, policies and environment dynamics, or message-passing components. Horizontal arrows encode evaluative structure, such as losses and gradients, reward and value assessments, norms and meta-objectives. The 2-cells are agentic components that couple behavior and evaluation, such as learning layers, RL agents, critics, overseers, and institutions.

Deep-learning architectures with their training loops, multi-agent RL systems with their interaction and credit assignment, and orchestrated multi-agent systems with tool use and oversight, are therefore expressed in the same agentic diagrammatic calculus. Category theory is not merely a generic language for models; it supplies a concrete two-dimensional structure, an agentic double category, in which both micro-scale deep learning architectures and macro-scale agentic systems can be described, compared and transformed by the same compositional operations. This is the formal core behind the conceptual position in Section 2.

4 Examples

4.1 Protocol-based Agentic Orchestration

We show that recent protocol-based agent orchestration frameworks instantiate the agentic double-category structure introduced in Section 3.1. Although such systems are typically not learning-based, they exhibit a clear separation between behavioural interaction and evaluative (normative) structure. In the present setting, evaluation is understood primarily in terms of *admissibility*, *authorization*, and *compliance*.

Behavioural category and the meaning of branching. Throughout this example, we fix the behavioural category $\mathcal{B} = \text{Stoch}$ as in Section 3.3, interpreting morphisms as stochastic interaction processes between agents. Protocol-induced branching and conditional behaviour are modeled in Stoch via Markov kernels. Importantly, this stochastic branching reflects behavioural nondeterminism (e.g. multiple possible outcomes of execution), rather than normative admissibility, which is treated separately by the evaluative structure. Objects are taken to be protocol-level interfaces X encoding communication states, task/session states, and message contexts.

Boolean evaluative structure (convention for this subsection). In this subsection we adopt the Boolean quantale

$$\mathcal{V} = \mathbf{2} = \{\perp, \top\}$$

to model protocol-level normative evaluation. Evaluative structure is represented by Boolean-valued profunctors

$$R : X^{\text{op}} \times X \longrightarrow \mathbf{2},$$

depicted as horizontal arrows in the sense of double categories. Intuitively, $R(x, x') = \top$ means that a behavioural transition from protocol state x to x' is admissible under the protocol, while $R(x, x') = \perp$ means rejection.

Both the Agent-to-Agent (A2A) protocol and the Agent Network Protocol (ANP) define structured agent interaction through tasks, messages, and negotiation or execution steps. In both cases, protocol execution gives rise to behavioural transitions in the vertical category $\mathcal{A}_v = \mathcal{B}$, but with distinct semantic roles.

A2A as a productive protocol. In A2A, protocol execution may *produce persistent protocol artefacts*, such as authorization claims or capability declarations derived from AgentCards. Accordingly, an execution step is naturally modeled as a behavioural morphism

$$f_{\text{A2A}} : X \longrightarrow X \otimes J,$$

where J is an interface collecting artefacts emitted by execution (e.g. claims, capability declarations, attestations, execution traces). These artefacts record evidence or outcomes of protocol interaction and may persist across subsequent steps. Crucially, they do not themselves constitute normative judgement; admissibility is instead captured by Boolean-valued evaluation.

Kernel-level semantics of artefact production in A2A. We make the behavioural content of f_{A2A} explicit at the level of Markov kernels. Regard X and J as measurable spaces. An A2A execution step is a Markov kernel

$$f_{A2A} : X \longrightarrow \mathcal{P}(X \otimes J).$$

We assume it factorizes into a stochastic state update and artefact emission:

$$\text{next}_X : X \longrightarrow \mathcal{P}(X), \quad \text{emit}_J : X \longrightarrow \mathcal{P}(J),$$

and define the behavioural transition by the product kernel

$$f_{A2A}(x) := \text{next}_X(x) \otimes \text{emit}_J(x).$$

To model heterogeneous artefacts, take J to be a disjoint union

$$J := J_{\text{task}} \sqcup J_{\text{cap}} \sqcup J_{\text{auth}},$$

corresponding to task artefacts, AgentCard-derived capability declarations, and authentication/authorization objects. For each class, fix an emission kernel

$$q_{\bullet} : X \longrightarrow \mathcal{P}(J_{\bullet}),$$

and measurable weights

$$\lambda_{\bullet} : X \longrightarrow [0, 1], \quad \sum_{\bullet} \lambda_{\bullet}(x) = 1.$$

Define the emission kernel by the convex combination

$$\text{emit}_J(x) := \lambda_{\text{task}}(x) q_{\text{task}}(x) + \lambda_{\text{cap}}(x) q_{\text{cap}}(x) + \lambda_{\text{auth}}(x) q_{\text{auth}}(x),$$

viewing the measures on the right as elements of $\mathcal{P}(J)$ via the canonical injections into the coproduct.

A2A evaluation on the extended interface. Since A2A enriches the interface to $Y := X \otimes J$, its post-execution admissibility is evaluated by a Boolean profunctor

$$R_{A2A} : Y^{\text{op}} \times Y \longrightarrow \mathbf{2}.$$

Example 1 (Protocol artefacts in A2A execution). *In A2A, execution steps generate persistent protocol artefacts recording factual outcomes of interaction. Typical examples include:*

- Task artefacts, such as intermediate results, partial outputs, or execution traces produced during a streamed task lifecycle.
- Capability declarations derived from AgentCards, specifying advertised skills, supported actions, or interface constraints.
- Authentication or authorization artefacts, such as tokens, claims, or attestations produced during verification or negotiation phases.

These artefacts belong to the emitted interface J and are produced behaviourally via $f_{A2A} : X \rightarrow X \otimes J$. Whether a subsequent behavioural transition is permitted is determined separately by Boolean evaluation on the enriched interface $Y = X \otimes J$.

ANP as a constraining protocol. In contrast, ANP emphasizes admissibility and negotiation. Here protocol judgements primarily act as *constraints* on behaviour, filtering or gating which interactions may proceed. Accordingly, the underlying behavioural step is more faithfully modeled as

$$f_{\text{ANP}} : X \longrightarrow X,$$

with negotiation outcomes encoded not as behavioural outputs, but by the Boolean evaluative structure and the associated agentic 2-cells. Any concrete negotiation traces or credentials, when retained, are treated as part of the protocol state X itself (not as a separate J).

Kernel-level semantics of gating and filtering in ANP. In Stoch, the map $f_{\text{ANP}} : X \rightarrow X$ is a kernel

$$f_{\text{ANP}} : X \longrightarrow \mathcal{P}(X).$$

We record two standard kernel-level formulations of gating.

(A) Absorbing-failure semantics. Extend the state space to

$$X_{\perp} := X \sqcup \{\perp\},$$

where $\perp$ denotes rejection/termination. Let $\text{adm} : X \rightarrow \mathbf{2}$ be an admissibility predicate and $\text{next} : X \rightarrow X$ a (deterministic or stochastic) proposal. Define

$$f_{\text{ANP}}(x) := \begin{cases} \delta_{\text{next}(x)}, & \text{if } \text{adm}(x) = \top, \\ \delta_{\perp}, & \text{if } \text{adm}(x) = \perp, \end{cases}$$

where δ_y is the Dirac measure at y .

(B) Constraint-induced conditioning. Alternatively, let $k : X \rightarrow \mathcal{P}(X)$ be a proposal kernel and $C : X \rightarrow \mathbf{2}$ a constraint predicate. Define the conditioned kernel

$$f_{\text{ANP}}(x)(A) := \frac{k(x)(A \cap C^{-1}(\top))}{k(x)(C^{-1}(\top))}, \quad \text{whenever the denominator is nonzero.}$$

This expresses gating as constraint-induced filtering of the behavioural distribution.

ANP evaluation on the internal interface. Unlike A2A, ANP does not extend the interface: all normative judgements act directly on X via a Boolean profunctor

$$R_{\text{ANP}} : X^{\text{op}} \times X \longrightarrow \mathbf{2}.$$

Example 2 (Admissibility and negotiation in ANP). *ANP is designed primarily as a constraining and negotiating protocol. At the identity and secure communication layer, agents authenticate using decentralized identifiers (DIDs) and cryptographic signatures. Tokens or credentials, when exchanged, function as access enablers that gate requests, rather than as persistent artefacts to be consumed by later protocol stages.*

At the meta-protocol layer, agents negotiate whether communication can proceed and under which protocol. The outcome (agreement, rejection, revision) acts as a constraint on which behavioural steps are permitted, rather than an artefact appended to the state.

At the application protocol layer, agent descriptions and discovery mechanisms expose capabilities and interfaces, while authorization mechanisms filter allowable behaviour. From the agentic perspective, these features are naturally modeled by $f_{\text{ANP}} : X \rightarrow X$ in the behavioural category, with admissibility and negotiation enforced by Boolean evaluation and agentic 2-cells.

Protocol norms as profunctorial evaluation (examples). The explicit normative mechanisms appearing in A2A and ANP naturally instantiate Boolean-valued evaluative profunctors. Typical examples include:

- **AgentCards and capability requirements:** a Boolean predicate evaluating whether a request context satisfies required capabilities.
- **Authentication/authorization checks:** Boolean evaluation of whether identities/credentials permit an action.
- **Security and trust constraints:** conjunction of constraints expressed as Boolean composition.
- **Negotiation rules:** Boolean evaluation of whether proposed protocol choices meet negotiated conditions.

Protocol enforcement as an agentic 2-cell. Protocol enforcement and compliance checking mechanisms act as an *agentic 2-cell*

$$A_{\text{proto}} : (R, f) \Rightarrow (R', g),$$

witnessing that behavioural evolution is coherent with respect to protocol-level evaluation and enforcement. The precise behavioural type depends on the protocol: in A2A, one typically has $f, g : X \rightarrow X \otimes J$ (artefact-producing steps), while in ANP one has $f, g : X \rightarrow X$ (gating/filtering steps).

Writing $Y := X \otimes J$ in the A2A case and $Y := X$ in the ANP case, the generic agentic square is

$$A_{\text{proto}} : \begin{array}{ccc} X & \xrightarrow{R} & X \\ f \downarrow & \Downarrow A_{\text{proto}} & \downarrow g \\ Y & \xrightarrow{R'} & Y \end{array}$$

where R and R' are Boolean-valued profunctors. Intuitively, the 2-cell certifies that the post-enforcement behaviour g is admissible relative to the updated evaluation, e.g. that $R'(y, g(y)) = \top$ for the relevant interface state y .

Example 3 (Protocol enforcement as agentic 2-cells).

• **A2A (*productive*):** enforcement validates a request against AgentCard capabilities and authorization rules, and successful execution may emit additional artefacts (claims/attestations), yielding a coherent transition $X \rightarrow X \otimes J$ certified by A_{proto} .

• **ANP (*constraining*):** enforcement performs admissibility checks (authentication, authorization, negotiated conditions) that gate whether the behavioural transition $X \rightarrow X$ may proceed; coherence is expressed by $R'(x, g(x)) = \top$.

• **Compositionality:** vertical composition models successive enforcement rounds over time, while horizontal composition models accumulation of multiple Boolean constraints.

In summary, A2A and ANP realize two structurally distinct protocol semantics: A2A is *productive* (behaviour may emit persistent artefacts), whereas ANP is *constraining* (behaviour is gated by admissibility and negotiation). The agentic double-category viewpoint makes this distinction explicit as a difference in the *behavioural interface evolution* ($X \rightarrow X \otimes J$ versus $X \rightarrow X$), while keeping the normative layer uniformly represented as Boolean-valued profunctorial evaluation certified by agentic 2-cells.

5 Conclusion

To be completed.

References

Roberto Bruni, José Meseguer, and Ugo Montanari. Symmetric monoidal and cartesian double categories as a semantic framework for tile logic. *Mathematical Structures in Computer Science*, 12(1):53–90, 2002.

Bruno Gavranović, Paul Lessard, Andrew Dudzik, Tamara Von Glehn, João G.M. Araújo, and Petar Veličković. Position: categorical deep learning is an algebraic theory of all architectures. In *ICML*, 2024.

A Four Canonical Agentic Examples

This appendix collects four canonical examples of the agentic double-category framework introduced in Section 3.

Example 4 (Supervised Learning and Neural Networks). *Supervised learning with neural networks provides the most basic instantiation of the agentic double-category framework. Although often*

presented as a purely optimization-driven process, standard training pipelines already exhibit a clear separation between behavioral evolution and evaluative structure.

Objects. Objects represent representational interfaces, such as input and output spaces, feature spaces, or intermediate latent representations of a model. At a fixed training stage, an object captures the semantic interface through which the model interacts with data and downstream tasks.

Vertical 1-cells (Behavior). Vertical 1-cells represent behavioral or learning processes that evolve the model over time. Typical examples include forward computation, parameter updates, layer-wise composition, and fine-tuning. Formally, a vertical morphism

$$f : X \longrightarrow Y$$

describes how a model with interface X is transformed into a model with interface Y through training or architectural modification. Such transformations are generally irreversible, reflecting the many-to-one nature of operations such as gradient descent, pruning, representation compression, or distillation.

Horizontal 1-cells (Evaluation). Horizontal 1-cells encode evaluative structure that assesses or constrains model behavior without itself generating new behavior. Typical examples include loss functions, risk functionals, and regularizers. A horizontal morphism

$$p : X \Rightarrow Y$$

assigns an evaluative relation between interfaces, measuring how model outputs at Y are judged relative to inputs or representations at X .

Agentic 2-cells. An agentic 2-cell

$$\alpha : (p, f) \Rightarrow (p', f')$$

witnesses the compatibility between behavioral evolution and evaluative structure. Semantically, such a 2-cell asserts that the learning update f implemented under the evaluation p yields a new behavior f' whose performance is coherently assessed by the updated evaluation p' . In practice, this corresponds to statements such as stochastic gradient descent updates are compatible with a given loss function, or fine-tuning preserves task-level performance. Importantly, the 2-cell does not certify optimality or reversibility; rather, it expresses admissibility or non-degradation of behavior under the chosen evaluation.

This example typically instantiates a Level 2 notion of compatibility, in which learning updates are irreversible but are constrained so as not to increase risk or uncertainty relative to the evaluative criteria.

Example 5 (Reinforcement Learning, Actor–Critic, and Multi-agent Systems). Reinforcement learning provides a canonical instantiation of the agentic double-category framework in which behavioral interaction and evaluation are both stochastic and tightly coupled. Unlike supervised learning, evaluation in reinforcement learning does not merely assess outcomes after the fact, but actively constrains and guides behavioral updates.

Objects. Objects represent interaction interfaces, such as an agent’s internal parameter or memory space, the environment state space, and their composite products. At a given time step, an object captures the joint interface through which the agent and environment interact.

Vertical 1-cells (Behavior). Vertical 1-cells represent stochastic behavioral processes describing agent–environment interaction and learning. Formally, a vertical morphism is given by a Markov kernel

$$F : \Theta \otimes S \longrightarrow \mathcal{P}(\Theta \otimes S \otimes R),$$

which maps the current agent parameters Θ and environment state S to a distribution over updated parameters, next states, and realized reward signals. This kernel typically arises from the composition of a policy, the environment dynamics, and a learning update, and is generally irreversible.

Horizontal 1-cells (Evaluation). Horizontal 1-cells encode evaluative or normative structure, such as reward functions, value functions, critics, or risk-sensitive constraints. They are modeled as enriched profunctors

$$R : (\Theta \otimes S)^{\text{op}} \times (\Theta \otimes S) \longrightarrow V,$$

assigning to each pair of pre- and post-interaction interfaces an evaluative value in a chosen quantale V . Crucially, these evaluative structures do not generate behavior themselves; instead, they assess and constrain behavioral evolution.

Agentic 2-cells. An agentic 2-cell

$$\alpha : (R, F) \Rightarrow (R', F')$$

witnesses the admissibility of a behavioral update relative to the evaluative structure. Semantically, it asserts that the transition from the behavioral process F to an updated process F' is coherent with respect to evaluation, as expressed by the corresponding transformation from R to R' . In actor-critic methods, for example, the critic is not a passive observer but participates in constraining the policy update, ensuring Bellman-consistent or otherwise admissible improvement steps.

In the multi-agent setting, objects and behavioral processes are formed by tensoring individual agents' interfaces and dynamics. The resulting 2-cells certify that joint policy updates are admissible under system-level evaluation, such as team reward, social welfare, or institutional constraints, rather than asserting individual optimality.

This example typically instantiates a transition from Level 2 to Level 3 compatibility: behavioral updates are irreversible and stochastic, and coherence with evaluation is expressed in an approximate or metric sense rather than by exact equivalence.

Example 6 (LLM-based Agentic Orchestration without Reinforcement Learning). Large language model (LLM) orchestration frameworks provide a particularly important instantiation of the agentic double-category framework in which agentic behavior is coupled to evaluation and normative structure without relying on reinforcement learning. Despite the absence of explicit reward maximization or policy-gradient updates, these systems exhibit the same two-dimensional separation between behavioral processes and evaluative constraints.

Objects. Objects represent protocol- and workflow-level interfaces, such as prompt and message states, tool input/output schemas, memory buffers, and intermediate workflow or planner states. An object captures the structured context through which an LLM-based agent interacts with tools, other agents, and external environments.

Vertical 1-cells (Behavior). Vertical 1-cells represent behavioral execution steps, including LLM invocations, tool calls, routing or planning decisions, and workflow transitions. Formally, a vertical morphism

$$f : X \longrightarrow \mathcal{P}(Y)$$

maps an input interface X to a (possibly stochastic) set or distribution of successor interfaces Y . Such nondeterminism may arise from sampling in LLM generation, tool execution outcomes (success, failure, retry), or branching decisions in planners and orchestrators. Vertical composition corresponds to workflow or call-graph composition, as implemented in directed acyclic graphs or cyclic execution pipelines.

Horizontal 1-cells (Evaluation). Horizontal 1-cells encode evaluative or normative structure, such as verifiers, rubrics, policy constraints, safety checks, or reward models used purely for assessment. They are modeled as enriched profunctors

$$R : X^{\text{op}} \times Y \longrightarrow V,$$

assigning evaluative values such as cost, risk, latency, compliance, or credibility to transitions between workflow interfaces. Importantly, these evaluative components do not drive behavior through reward optimization; instead, they act as external constraints, filters, or certification mechanisms.

Agentic 2-cells. An agentic 2-cell

$$\alpha : (R, f) \Rightarrow (R', g)$$

witnesses that a modification of behavior, such as a workflow rewrite, planner revision, or tool substitution, is admissible under the corresponding evaluative structure. Semantically, the 2-cell asserts that replacing the execution step f by g does not violate the relevant norms or constraints, for example by keeping cost below a threshold, maintaining compliance, or reducing risk.

Typical instances include offline evaluation protocols, runtime policy gates, and certified workflow rewrites, such as inserting verification steps, replacing tools with safer alternatives, or restructuring call graphs to reduce latency. Crucially, no reinforcement-learning signal is used to optimize behavior; evaluation serves only to accept, reject, or constrain behavioral modifications.

This example instantiates a Level 3 notion of compatibility, in which behavioral processes are highly irreversible and stochastic, and coherence with evaluation is expressed in an approximate or metric-dependent manner rather than through exact equivalence or optimization.

Example 7 (Symbolic Reasoning, Planning, and Formal Inference). *Symbolic reasoning and formal planning provide an instantiation of the agentic double-category framework at the opposite end of the spectrum from stochastic learning-based systems. In this setting, behavioral processes are typically discrete, rule-based, and highly structured, and compatibility with evaluation often approaches exact equivalence.*

Objects. *Objects represent symbolic or logical interfaces, such as logical states, planning configurations, proof contexts, or abstract syntax trees. An object captures the formal structure through which symbolic agents interact with specifications, constraints, and reasoning rules.*

Vertical 1-cells (Behavior). *Vertical 1-cells represent symbolic transformations, such as rule applications, proof steps, program rewrites, or planning transitions. Formally, a vertical morphism*

$$f : X \longrightarrow Y$$

describes the application of a syntactic or semantic transformation that takes one symbolic configuration to another. Unlike learning-based updates, these transformations are typically deterministic and, in many cases, reversible up to renaming or structural equivalence.

Horizontal 1-cells (Evaluation). *Horizontal 1-cells encode formal correctness criteria, specifications, or logical constraints. They assess whether a symbolic transformation preserves validity, satisfies a specification, or respects a given set of rules. Such evaluative structure does not generate new symbolic behavior, but instead judges behavioral steps with respect to formal norms.*

Agentic 2-cells. *An agentic 2-cell*

$$\alpha : (p, f) \Rightarrow (p, g)$$

witnesses that two behavioral transformations are compatible with the same evaluative structure. Semantically, it asserts that a reasoning or planning step is correct with respect to the specification, and therefore interchangeable with another step or sequence of steps. Typical instances include proofs of correctness, commuting conversions, normalization witnesses, or certified planning refinements.

In contrast to learning-based examples, the 2-cells in this setting often express near-equivalence rather than mere admissibility. As a result, symbolic reasoning systems typically instantiate a high-certainty core in which compatibility is close to reversible, modulo inessential syntactic variation.

This example corresponds to the highest end of the compatibility spectrum, where coherence between behavior and evaluation approaches exact correctness rather than approximate or metric-based agreement.

B Design Axioms and Well-Formedness of the Agentic Double Category

This appendix records the design axioms underlying the agentic double-category semantics used throughout the paper. These axioms are not additional modeling assumptions, but rather well-formedness conditions ensuring that the double-category structure faithfully reflects the operational constraints of protocol-based agent systems.

Let $\mathbb{A}$ be a strict agentic double category whose objects represent protocol-level agent interfaces or roles, whose vertical 1-cells represent concrete agent behaviors, whose horizontal 1-cells represent institutional, evaluative, or protocol constraints, and whose 2-cells witness admissible behavioral change.

Axiom A.1 (AI-Native Interaction). All vertical 1-cells in $\mathbb{A}$ operate on agent-native semantic interfaces. Objects corresponding to human-facing artifacts (e.g. GUIs, rendered webpages) are excluded from the object class of $\mathbb{A}$.

Counterexample. Agent interaction mediated exclusively by browser automation.

Failure. Vertical composition becomes unstable and non-compositional.

Axiom A.2 (Protocol Compatibility). Every admitted communication protocol must be representable as a horizontal 1-cell or as a constrained family of vertical 1-cells. Opaque, non-semantic interfaces are disallowed.

Counterexample. A proprietary RPC interface without semantic description.

Failure. Horizontal structure becomes partially invisible, preventing the formation of 2-cells.

Axiom A.3 (Compositional Modularity). Vertical and horizontal 1-cells must be independently composable. Protocol components may not be intrinsically monolithic.

Counterexample. Identity, discovery, and execution fused into a single protocol step.

Failure. The double category collapses toward a single-category structure.

Axiom A.4 (Minimal Core with Extensibility). Core vertical semantics remain fixed, while extensions are expressed through horizontal refinements or additional 2-cells.

Counterexample. Introducing new capabilities by modifying execution semantics.

Failure. 2-cell comparability and backward compatibility are lost.

Axiom A.5 (Pragmatic Deployability). All objects and morphisms must correspond to deployable engineering entities. Idealized global coordination is excluded.

Counterexample. A globally synchronized, universally trusted agent registry.

Failure. The semantics admit no faithful implementation.

Axiom A.6 (Least Trust). Every vertical 1-cell is subject to at least one horizontal constraint, and admissibility is witnessed by explicit 2-cells.

Counterexample. Unauthenticated high-risk agent actions.

Failure. The horizontal dimension collapses, eliminating accountability and auditability.

Together, these axioms ensure that the agentic double category is not merely formally definable, but operationally meaningful and compositional.